WNDRVR
SUPPORT NETWORK

VXWORKS 7 KERNEL APPLICATION TUTORIAL - WORKBENCH 4

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Kernel Application Tutorial - Workbench 4

27 August 2020

1. KERNEL APPLICATION CREATION USING WORKBENCH 4

The VxWorks® 7 Kernel Application Tutorial - Workbench 4 shows you how to create, build, run, and debug a kernel application using Workbench 4.

VxWorks applications that execute in the kernel are created as relocatable object modules and are executed as a set of tasks running in kernel space. They can specifically be referred to as kernel-based application modules, but they are often called kernel application modules or kernel applications. In VxWorks, they are referred to as downloadable kernel modules (DKMs), even when they are linked with the kernel. A kernel application is derived from a DKM project that includes user source code.

To create a DKM project, you must first create a VxWorks source build (VSB) project, which generates binary kernel libraries based on a target architecture. You must then create a VxWorks image project (VIP) based on the VSB. The VIP lets you configure and build a VxWorks image. You can then create a DKM project based on the VSB. The DKM uses the binary kernel libraries generated from the VSB.

In this tutorial, you will perform the following tasks:

1. Create and build a VSB.
2. Create and build a VIP based on the VSB.
3. Create a DKM project based on the VSB, add code to the project, and build the project to obtain a kernel application.
4. Connect to the VxWorks simulator and boot the VxWorks kernel image.
5. Load and run the kernel application.
6. Debug the kernel application in application mode.

WNDRVR

2. CONFIGURING AND BUILDING VXWORKS SOURCE

To generate VxWorks kernel libraries, you must create, configure, and build a VxWorks source build (VSB) project.

About This Task

For this tutorial, you will create a VSB based on a board support package for the VxWorks simulator.

Procedure

1. Start Workbench and, in the Workspace Launch dialog box, click **Launch**.

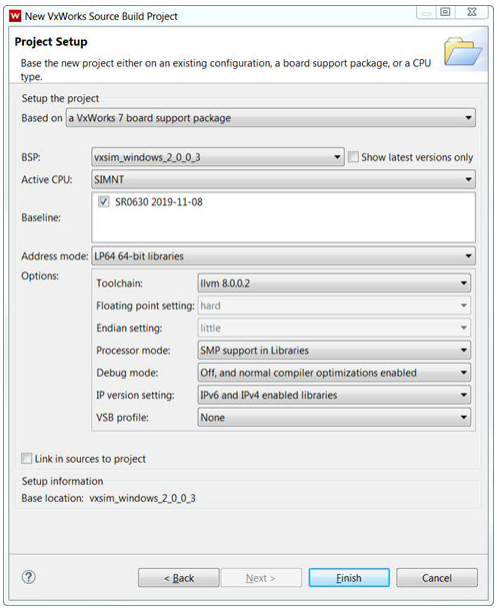
   This creates a workspace folder in the specified directory (*installDir*/*workspaceDir*) if the folder does not already exist.

2. Ensure that Workbench is in the System Development perspective.

   The System Development perspective contains a set of views for creating VxWorks native and user-defined projects, browsing source code, and creating target connections.

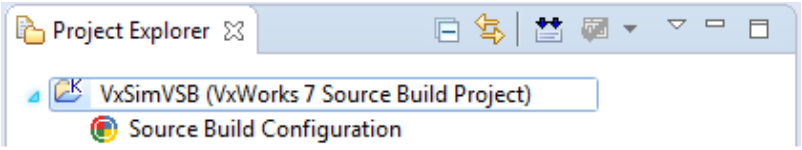
3. From the Workbench menu, select **File > New > Wind River Workbench Project**.
4. From the **Build type** drop-down menu, select **VxWorks Source Build** and click **Next**.
5. In the **Project name** field, type **VxSimVSB**, and then click **Next**.
6. Configure the VSB project options:
   1. From the **Based-on** drop-down menu, select **a VxWorks 7 board support package**.
   2. From the **BSP** drop-down menu, select the VxWorks simulator BSP for your host type.

Host OS	BSP
Windows	**vxsim_windows**_*x*_*x*_*x*_*x*
Linux	**vxsim_linux**_*x*_*x*_*x*_*x*



7. Keep the remaining default settings and click **Finish**.

   The **VxSimVSB** project appears in the Project Explorer view.



8. Examine the **VxSimVSB** project configuration:
   1. In the Project Explorer view, expand the **VxSimVSB** project, right-click **Source Build Configuration**, and select **Edit Source Build Configuration**.
   2. In the Source Build Configuration view, right-click any option and select **Find**.
   3. In the **Pattern** field, type **debug**.

   In the Matching panel, the **DEBUG** option is set to **Y**. This option is enabled by default to provide debugging libraries.

   For this tutorial, do not edit any default configuration options.

9. In the Project Explorer view, right-click the **VxSimVSB** project and select **Build Project**.

   The Build Console view displays the build process.

3. CONFIGURING AND BUILDING A VXWORKS KERNEL IMAGE

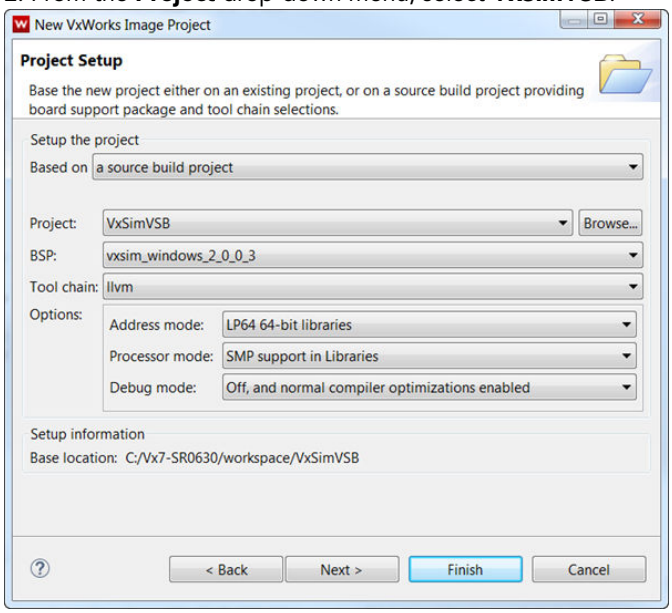
To create a VxWorks kernel image, you must create, configure, and build a VxWorks image project (VIP) based on a VxWorks source build (VSB) project.

About This Task

For this tutorial, you will create a VIP based on the **VxSimVSB** project that you created in Configuring and Building VxWorks Source on page 2.

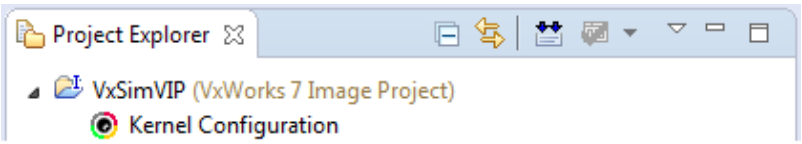
Procedure

1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
2. From the **Build type** drop-down menu, select **VxWorks Image Project** and click **Next**.
3. In the **Project name** field, type **VxSimVIP**, and then click **Next**.
4. Configure the VIP project options:
    1. From the **Based on** drop-down menu, select **a source build project**.
    2. From the **Project** drop-down menu, select **VxSimVSB**.



    3. Keep the remaining default settings, and click **Finish**.

The **VxSimVIP** project appears in the Project Explorer view.



5. Expand the **VxSimVIP** project, right-click the **Kernel Configuration** node, and select **Edit Kernel Configuration**.
6. Configure the **VxSimVIP** project with support for application mode debugging:
    1. In the Kernel Configuration editor, right-click any component and select **Find**.

2. In the **Pattern** field, type **debug_agent**.
3. Ensure that the **INCLUDE_DEBUG_AGENT** and **INCLUDE_DEBUG_AGENT_START** components are included.

If the components are not in bold, they are not included in the VIP. For each excluded component, you must select the component, click **Find**, right-click the component, and select **Include (Quick Include)**.

The **INCLUDE_DEBUG_AGENT** component allows you to debug in application mode. The **INCLUDE_DEBUG_AGENT_START** component starts the debug agent when the system boots.

7. In the Project Explorer view, right-click the **VxSimVIP** project and select **Build Project**.

   Observe the build process from the Build Console tab.

   After the build completes, the kernel image resides in the **default** folder. For example:

   installDir/*workspaceDir* **/VxSimVIP/default/vxWorks**

WNDRVR

4. CREATING AND BUILDING A KERNEL APPLICATION

To create a kernel application, you must create a downloadable kernel module (DKM) project based on a VxWorks source build (VSB) project, add source code, and build the project.

About This Task

For this tutorial, you will base the DKM project on the **VxSimVSB** project that you created in Configuring and Building VxWorks Source on page 2.

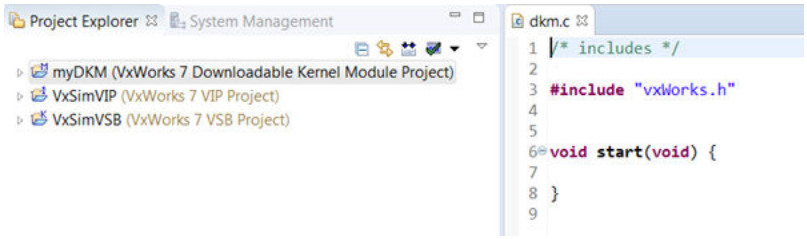
You will add code to a stub C file to create a kernel application that displays the following output on the console:

```
Hello World!
From the kernel...
```

Procedure

1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
2. From the **Build type** drop-down menu, select **Downloadable Kernel Module** and click **Next**.
3. In the **Project name** field, type **myDKM**, and then click **Next**.
4. Configure the DKM project options:
   1. From the **Based on** drop-down menu, select **a source build project**.
   2. From the **Project** drop-down menu, select **VxSimVSB**.
   3. Click **Finish**.

   The **myDKM** project appears in the Project Explorer view. Workbench creates and opens a stub C source file named **dkm.c** in the project.



   The stub C source file resides in the project folder. For example:

   installDir/*workspaceDir*/**myDKM**

5. In the **dkm.c** file, add a **stdio.h** include directive and a call to printf( ) as shown below.

```
/* includes */

#include "vxWorks.h"
#include <stdio.h>

void start(void) {
    printf("Hello World!\nFrom the kernel...\n");
}
```

6. Save the file.

7. In the Project Explorer view, right-click the **myDKM** project and select **Build Project**.

   If this is the first time the DKM project is being built, Workbench prompts you to edit the header file search path.

   1. Click **Generate Includes**.
   2. In the Generate Include Search Paths dialog box, select **Ignore all system include directives**, leave the other default setting, and click **Next**.
   3. If prompted, click **Next** to include search paths.
   4. Click **Finish**.
   5. In the C/C++ Index configuration changed dialog box, click **Yes** to rebuild the index and the DKM project.
8. Observe the build process from the Build Console tab.

5. LAUNCHING THE VXWORKS SIMULATOR

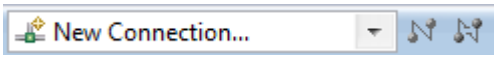
You must connect to the VxWorks simulator (VxSim) and boot a VxWorks kernel image.

About This Task

For this tutorial, you will boot the VxWorks kernel image that you created in Configuring and Building a VxWorks Kernel Image on page 4.

Procedure

1. From the Workbench toolbar, select **New Connection**.

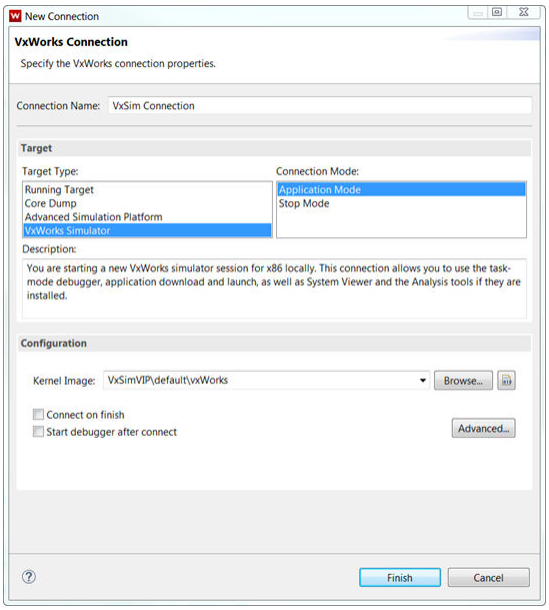


2. In the **Connection Name** field, type **VxSim Connection**.
3. In the Target panel, set the **Target Type** to **VxWorks Simulator** and the **Connection Mode** to **Application Mode**.
4. In the Configuration panel, click **Browse** and select the following kernel image.

   installDir/*workspaceDir*/**VxSimVIP/default/vxWorks**

   This is the kernel image that you created in Configuring and Building a VxWorks Kernel Image on page 4.

5. Clear the **Connect on finish** and **Start debugger after connect** check boxes.



6. From the Configuration panel, click **Advanced** and observe that the **Network Config** field is set to **NAT**—network address translation.

   The NAT configuration automatically redirects a number of ports by default. When you start the simulator, the default port redirections are printed to the console. This type of connection allows access to network resources and does not require a host connection driver or the VxWorks network daemon, **vxsimnetd**. By default, it acts as a firewall and does not permit any incoming traffic. It supports target communication framework (TCF) and user datagram protocol (UDP). For more information on the VxWorks network daemon, see the VxWorks 7 VxSim User's Guide.

7. Click **OK**.
8. Click **Finish** to save the simulator configuration as **VxSim Connection**.

   The VxSim Connection view appears.

   The tabs at the bottom of the VxSim Connection view display information relevant to an active session.

Details

Shows the Target and Configuration panels

File System

Displays the file system on the target

System Monitor

Allows you to browse the target system contexts (such as kernel tasks and RTPs)

Symbol Files

Allows you to browse the symbol files currently associated with each memory context

If a symbol file was not resolved automatically, locate the right symbol file by clicking **Locate Symbol File**.

Object Paths

Contains the path mapping rules applied to the target module

These mappings are used by the tools to automatically locate the symbol files for a given target module (DKM or RTP). Some entries are generated automatically as a result of the kernel symbol file location workflow, or any symbol file locate actions.

Source Paths

Contains the path mapping rules applied to locate source code

These mappings are used by the tools to locate the source files on the host from the path information embedded in the debug information.

In the Workbench toolbar, **VxSim Connection** is displayed as the network connection,

9. Next to **VxSim Connection**, click the **Connect** button to start the simulator and connect to the target.

   A green indicator appears next to the connection name in the Workbench tool bar. This indicates a successful connection to the target.

   The Terminals view displays a VxSim Connection view that shows the memory configuration, the default port redirections, and the VxWorks kernel shell prompt. For example:

WNDRVR

```
Checking License...OK
Virtual  Base Address: 0x0000000180000000
Virtual  Top  Address: 0x0000000500000000    Virtual  Size: 0x0000000380000000 (14Gb)
Physical Base Address: 0x0000000180000000
Physical Top  Address: 0x00000001a0000000    Physical Size: 0x0000000020000000 (512Mb)
Loading...
0x487288 + 0x72260
Redirecting TCP port 52058 to 10.0.10.2:21
Redirecting TCP port 52059 to 10.0.10.2:22
Redirecting TCP port 52060 to 10.0.10.2:23
Redirecting TCP port 52061 to 10.0.10.2:69
Redirecting TCP port 52062 to 10.0.10.2:80
Redirecting TCP port 52063 to 10.0.10.2:443
Redirecting TCP port 52064 to 10.0.10.2:1534
Redirecting UDP port 59747 to 10.0.10.2:17185
VxWorks CPU number: 2 / Host CPU number: 4
Instantiating /ram0 as rawFs,  device = 0x1
Formatting /ram0 for DOSFS
Instantiating /ram0 as rawFs, device = 0x1
Formatting...Retrieved old volume params with %38 confidence:
Volume Parameters: FAT type: FAT32, sectors per cluster 0
  0 FAT copies, 0 clusters, 0 sectors per FAT
  Sectors reserved 0, hidden 0, FAT sectors 0
  Root dir entries 0, sysId (null)  , serial number 60000
  Label:"           " ...
Disk with 16384 sectors of 512 bytes will be formatted with:
Volume Parameters: FAT type: FAT16, sectors per cluster 2
  2 FAT copies, 8143 clusters, 32 sectors per FAT
  Sectors reserved 1, hidden 0, FAT sectors 64
  Root dir entries 512, sysId VXDOS16 , serial number 60000
  Label:"           " ...
OK.

 ________           ________
 \77777777\          /77777777/
  \77777777\        /77777777/
   \77777777\      /77777777/
    \77777777\    /77777777/
     \77777777\   \7777777/
      \77777777\   \77777/              VxWorks 7 SMP 64-bit
       \77777777\   \777/
        \77777777\   \7/     Core Kernel version: 3.1.1.0
         \77777777\   -      Build date: Oct  3 2019 15:19:00
          \77777777\
           \7777777/         Copyright Wind River Systems, Inc.
            \77777/   -                 1984-2019
             \777/   /7\
              \7/   /777\
               -   -------

                   Board: SIMNT board
               CPU Count: 2
          OS Memory Size: 508MB
        ED&R Policy Mode: Deployed
     Debug Agent: Started (always)
         Stop Mode Agent: Started (always)
```

```
Loading symbol table from host:C:/Vx7-SR0630/workspace/VxSimVIP/default/vxWorks.sym ...done

->
```

To view the available commands, type help at the kernel shell prompt.

To reboot the instance, type reboot at the kernel shell prompt.

To terminate the connection, click the **Disconnect** button on the tool bar. To reconnect, click the **Connect** button.

To terminate the simulation, click the **Disconnect** button and close the **VxSim Connection** tab.

To relaunch the simulator instance and reconnect, click the **Connect** button.

WNDRVR

6. LOADING AND RUNNING THE KERNEL APPLICATION

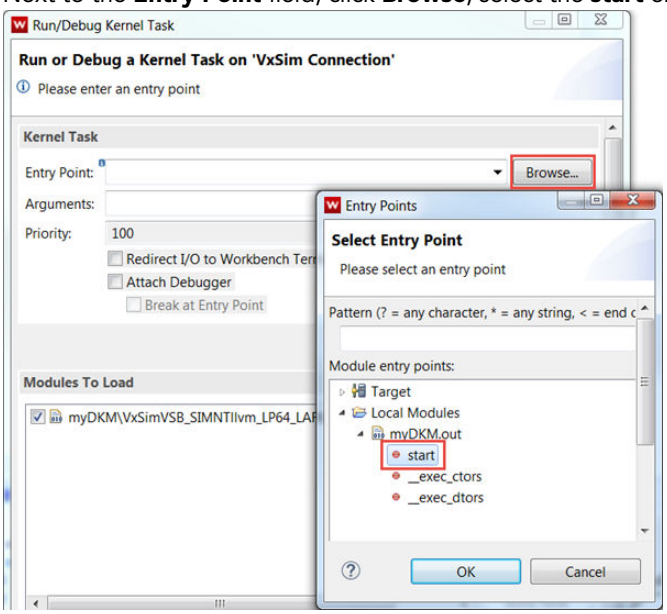
To run the kernel application, you must load and run the downloadable kernel module (DKM) on the VxWorks kernel.

Procedure

1. In the Project Explorer view, right-click the **myDKM** project and select **Run/Debug Kernel Task**.

   The Run/Debug Kernel Task dialog box appears.

2. Next to the **Entry Point** field, click **Browse**, select the **start** entry point, and click **OK**.



3. Click **OK**.

   The program loads and executes in kernel space.

4. Below the VxSim Connection view, click the **Terminal** tab.

   The console displays the following output.

```
Hello World!
From the kernel...
```

5. In the Workbench toolbar, click the **Disconnect** icon to disconnect from the target.

7. DEBUGGING A KERNEL APPLICATION

To debug a kernel application, you must launch the VxWorks Simulator (VxSim), run the DKM with the debugger attached, and apply VxWorks debugging features.

About This Task

In this tutorial you will debug the **myDKM** kernel application that you created in Creating and Building a Kernel Application on page 6. You will explore application mode debugging using code stepping and breakpoints.

Procedure

1. Add code to the **dkm.c** file:
   1. In the Project Explorer view, under the **myDKM** project, right-click the **dkm.c** file and select **Open**.
   2. Add the include and the definition shown in bold below under the existing includes.

```
#include "vxWorks.h"
#include <stdio.h>
#include <taskLib.h>

#define PREF "\033["
```

   3. Add the code shown in bold below under the printf( ) function.

```
printf("Hello World!\nFrom the kernel...\n");

printf("\n\n");
        printf("/--------------------\\\n");
        printf("|                    |\n");
        printf("\\--------------------/\n");
        printf("%s2%c",PREF,'A');
        printf("%s1%c",PREF,'C');
        putchar('#');
        fflush(stdout);
        taskDelay(5);
        char dir=0;
        while(1){
            int i;
            for (i=0; i<19;i++){
                printf("%s1%c",PREF,'D');
                putchar(' ');
                fflush(stdout);
                if (dir){
                    printf("%s2%c",PREF,'D');
                    fflush(stdout);
                }
                putchar('#');
                fflush(stdout);
                taskDelay(5);
            }
            if (dir) dir=0;
            else dir=1;
        }
```

   4. In the Workbench menu, click the **Save** icon.

5. In the Project Explorer view, right-click the **myDKM** project and select **Rebuild**.

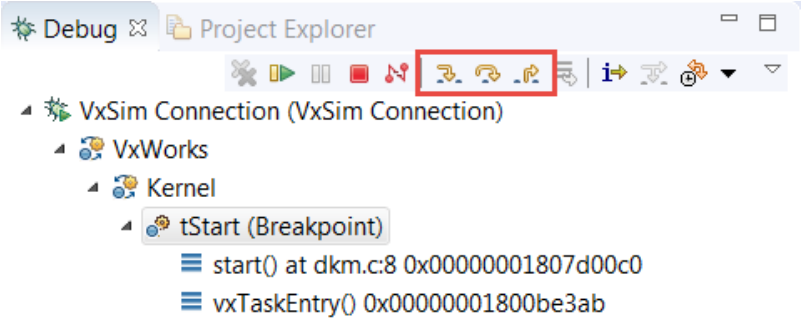
2. In the Workbench toolbar, click the **Connect** icon to launch VxSim.

3. Run the **myDKM** application:
   1. In the Project Explorer view, right-click the **myDKM** project and select **Run/Debug Kernel Task**.
   2. In the Run/Debug Kernel Task dialog box, configure the following settings:
      - Ensure that the **Entry Point** field contains **start**.
      - Select the **Attach Debugger** and **Break at Entry Point** check boxes.
   3. Click **OK**.

   Workbench switches to the Debug perspective, a temporary breakpoint is set at the entry point, and the application runs up to the entry point and is then suspended.

4. Near the bottom of Workbench, click the **Terminal** tab to view the kernel shell.

5. Step through the code from the Debug view:



   1. Click the **Step Over** icon six times to step over the first few of lines of code without going into subroutines.

   The console displays a frame in the output.

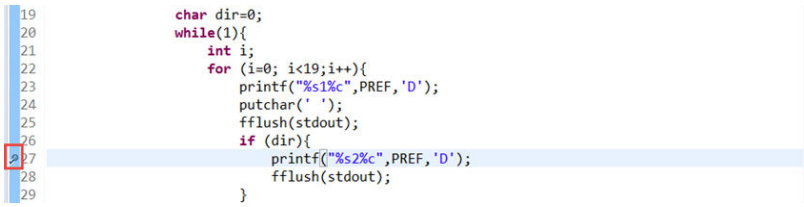
```
/--------------------\
|                    |
\--------------------/
```

2. Click the **Step Into** icon a few times to step through a subroutine.

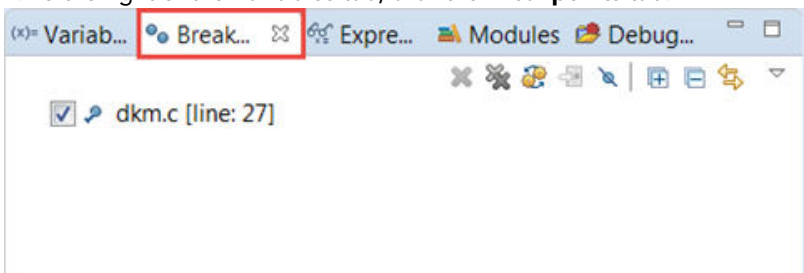
3. Click the **Step Return** icon to continue execution until the current subroutine completes and then return control to the debugger in the calling statement.

6. Set a breakpoint:
   1. In the **dkm.c** source file, double-click the gutter to the left of the following line.

```
printf("%s1%c",PREF,'D');
```



   2. To the right of the **Variables** tab, click the **Breakpoints** tab.



   The breakpoint that you added appears in the Breakpoints view.

   3. In the Debug view, click the **Resume** icon to let the application run until the next breakpoint.

   In the Terminal view, the console displays a hash character moving through the generated frame. Execution stops at the line with the breakpoint.

7. To the left of the **Breakpoints** tab, click the **Variables** tab.
8. Click the **Resume** icon twice.

   You hit the breakpoint each time you click the **Resume** icon. In the Terminal view, the hash character stops after each move. In the Variables view, the loop counter increments.

9. From the Workbench toolbar, click the **Disconnect** icon.

8. WHERE TO NEXT?

After creating and adding code to a VxWorks application using command line, based on a tutorial, see the following steps to connect to your target, load your DKM and call C functions in the kernel.

1. To connect to a remote file system see *VxWorks 7 File Systems and I/O Programmer's Guide: Accessing Remote File Systems from VxWorks*.
2. To load a DKM object see *VxWorks 7 Kernel Shell User's Guide:Loading C Modules into the Kernel*.
3. To call C functions see *VxWorks 7 Kernel Shell User's Guide: Calling C Functions From the C Interpreter*.

For more information on making a target connection and debugging with the WRDBG (Wind River DeBuG) shell see the second Debug Session Example in *WRDBG Debug Shell Reference: Getting Started*

For more information on VxWorks projects, see the *VxWorks 7 Configuration and Build Guide* and the *Kernel Application and Real-Time Process Application Projects Guide*.

For more information on getting started with VxWorks 7, see the getting started documentation from the Wind River Support Network at Getting Started.